SQL (Parte I)

Clase 03

IIC2413 - Sección 3

Prof. Cristian Riveros

Outline

Sobre SQL

Data Definition Language

Data Manipulation Language

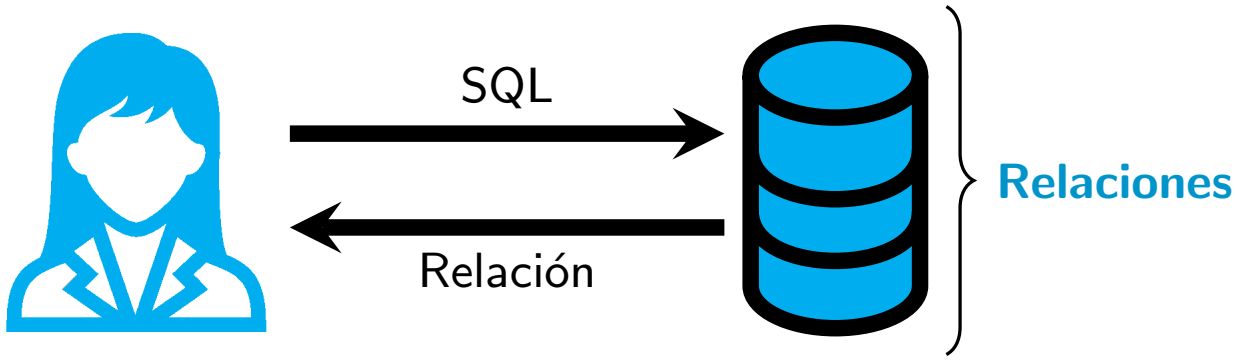
Data Query Language

Outline

¿qué es un RDBMS?

Definición

Un **sistema de manejo de bases de datos relacional** (RDBMS) es un software (**motor de BD**) encargado del almacenamiento, gestión, y optimización del acceso a los datos organizados en **relaciones**.



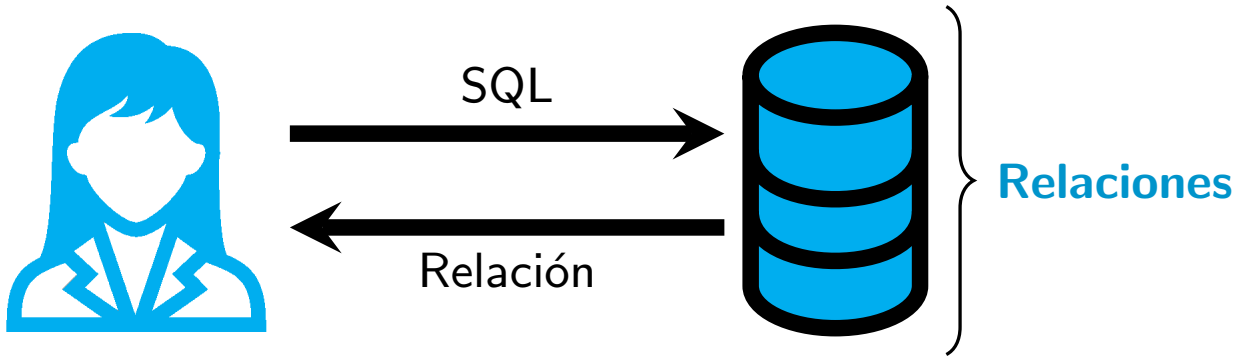
Ejemplos

- Arquitectura cliente-servidor
- Arquitectura en tres niveles (3-Tier)

¿qué es un RDBMS?

Definición

Un **sistema de manejo de bases de datos relacional** (RDBMS) es un software (**motor de BD**) encargado del almacenamiento, gestión, y optimización del acceso a los datos organizados en **relaciones**.



SQL es el lenguaje de acciones al RDBMS

DB-Engines

☑ include secondary database models

187 systems in ranking, August 2026

Rank			DBMS	Database Model	Score		
Aug 2026	Jul 2026	Aug 2025			Aug 2026	Jul 2026	Aug 2025
1.	1.	1.	Oracle	Relational, Multi-model	1123.43	-8.94	-97.27
2.	2.	2.	MySQL	Relational, Multi-model	842.32	-4.14	-73.15
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model	694.56	-4.40	-59.59
4.	4.	4.	PostgreSQL	Relational, Multi-model	684.58	-3.23	+13.32
5.	5.	5.	Snowflake	Relational	215.89	-0.17	+37.00
6.	6.	↑ 7.	Databricks	Multi-model	165.81	+1.73	+49.99
7.	7.	↓ 6.	IBM Db2	Relational, Multi-model	112.07	-0.92	-15.24
8.	8.	8.	SQLite	Relational	95.94	+1.17	-16.65
9.	9.	9.	MariaDB	Relational, Multi-model	79.08	-0.99	-14.52
10.	↑ 11.	↑ 12.	Apache Hive	Relational	71.09	+0.24	+0.05
11.	↓ 10.	11.	Microsoft Azure SQL Database	Relational, Multi-model	70.77	-3.79	-5.07
12.	12.	↓ 10.	Microsoft Access	Relational	64.30	-1.04	-23.46
13.	13.	13.	Google BigQuery	Relational	52.70	-0.38	-12.48
14.	14.	↑ 16.	SAP HANA	Relational, Multi-model	31.55	-1.16	-0.75
15.	15.	↓ 14.	Teradata	Relational, Multi-model	31.22	-0.75	-2.53
16.	16.	↓ 15.	FileMaker	Relational	29.39	-0.13	-3.93
17.	17.	17.	SAP Adaptive Server	Relational, Multi-model	25.10	-0.22	-3.04

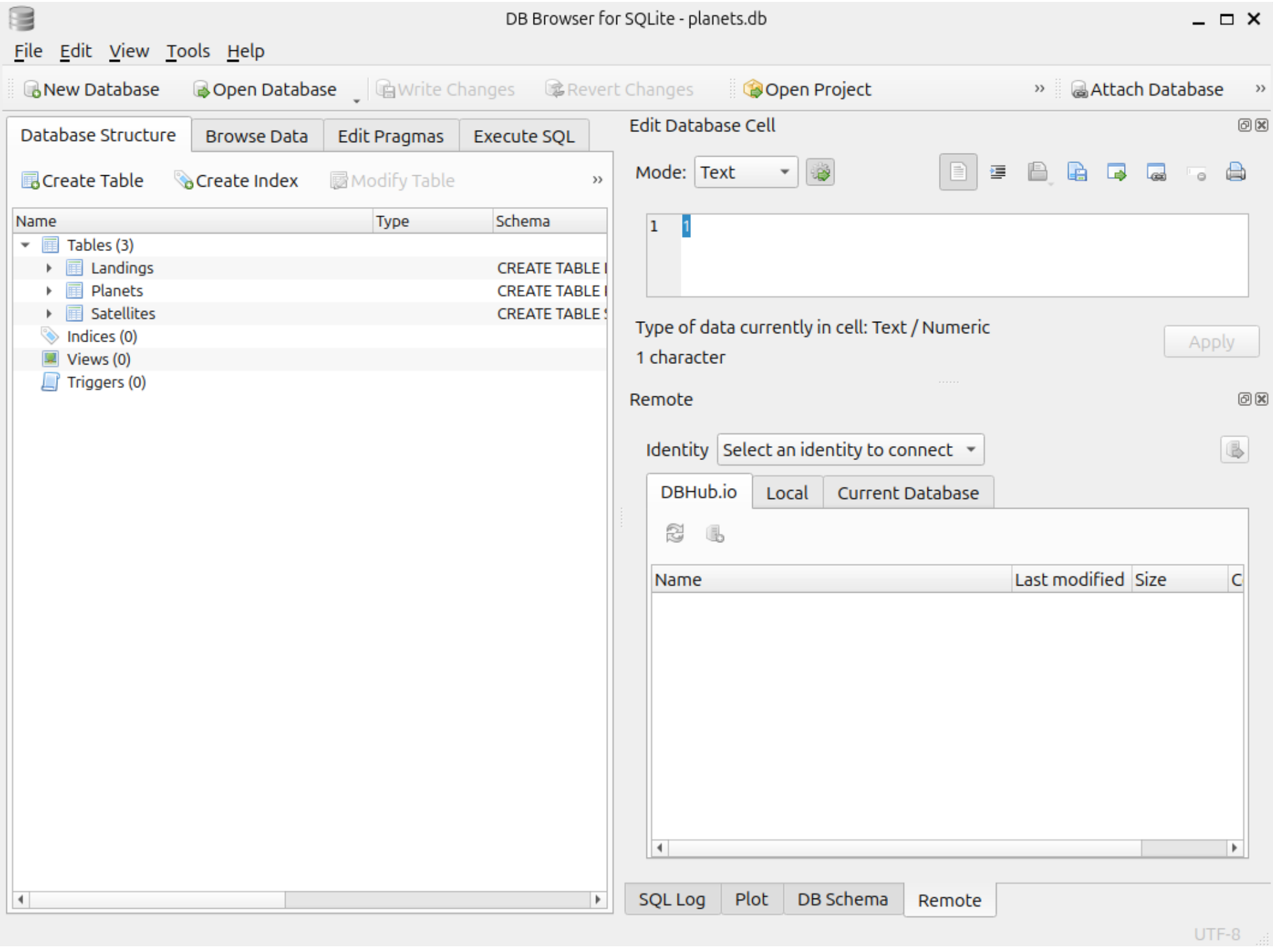
```
cristian@CRJ-X13-Yoga:~/IIC2413$
```

```
cristian@CRJ-X13-Yoga:~/IIC2413$ sudo apt install sqlite3
```

```
cristian@CRJ-X13-Yoga:~/IIC2413$ sudo apt install sqlite3
[sudo] password for cristian:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
sqlite3 is already the newest version (3.45.1-1ubuntu2.7).
The following packages were automatically installed and are no longer required:
  adwaita-qt docutils-common fonts-hack furo libadwaitaqt1 libadwaitaqtpriv1
  libgl1-amber-dri libglapi-mesa libllvm17t64 libqt5concurrent5t64
  libqt5multimedia5 libqt5multimediawidgets5 libqt5x11extras5 libqtermwidget5-1
  libquazip1-qt5-1t64 python3-alabaster python3-bs4 python3-cssselect
  python3-docutils python3-html5lib python3-imagesize python3-lxml
  python3-netifaces python3-roman python3-snowballstemmer python3-soupsieve
  python3-sphinx python3-sphinx-inline-tabs python3-webencodings
  qtermwidget5-data sphinx-basic-ng sphinx-common texstudio-doc
Use 'sudo apt autoremove' to remove them.
0 to upgrade, 0 to newly install, 0 to remove and 80 not to upgrade.
cristian@CRJ-X13-Yoga:~/IIC2413$
```

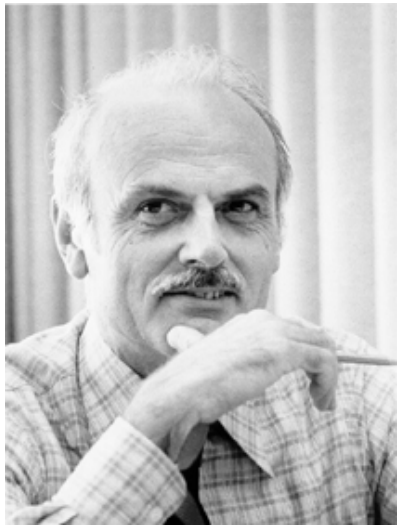
```
cristian@CRJ-X13-Yoga:~/IIC2413$ sudo apt install sqlite3
[sudo] password for cristian:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
sqlite3 is already the newest version (3.45.1-1ubuntu2.7).
The following packages were automatically installed and are no longer required:
  adwaita-qt docutils-common fonts-hack furo libadwaitaqt1 libadwaitaqtpriv1
  libgl1-amber-dri libglapi-mesa libllvm17t64 libqt5concurrent5t64
  libqt5multimedia5 libqt5multimediawidgets5 libqt5x11extras5 libqtermwidget5-1
  libquazip1-qt5-1t64 python3-alabaster python3-bs4 python3-cssselect
  python3-docutils python3-html5lib python3-imagesize python3-lxml
  python3-netifaces python3-roman python3-snowballstemmer python3-soupsieve
  python3-sphinx python3-sphinx-inline-tabs python3-webencodings
  qtermwidget5-data sphinx-basic-ng sphinx-common texstudio-doc
Use 'sudo apt autoremove' to remove them.
0 to upgrade, 0 to newly install, 0 to remove and 80 not to upgrade.
cristian@CRJ-X13-Yoga:~/IIC2413$ sqlite3 myfirstdatabase.db
```

```
cristian@CRJ-X13-Yoga:~/IIC2413$ sudo apt install sqlite3
[sudo] password for cristian:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
sqlite3 is already the newest version (3.45.1-1ubuntu2.7).
The following packages were automatically installed and are no longer required:
  adwaita-qt docutils-common fonts-hack furo libadwaitaqt1 libadwaitaqtpriv1
  libgl1-amber-dri libglapi-mesa libllvm17t64 libqt5concurrent5t64
  libqt5multimedia5 libqt5multimediawidgets5 libqt5x11extras5 libqtermwidget5-1
  libquazip1-qt5-1t64 python3-alabaster python3-bs4 python3-cssselect
  python3-docutils python3-html5lib python3-imagesize python3-lxml
  python3-netifaces python3-roman python3-snowballstemmer python3-soupsieve
  python3-sphinx python3-sphinx-inline-tabs python3-webencodings
  qtermwidget5-data sphinx-basic-ng sphinx-common texstudio-doc
Use 'sudo apt autoremove' to remove them.
0 to upgrade, 0 to newly install, 0 to remove and 80 not to upgrade.
cristian@CRJ-X13-Yoga:~/IIC2413$ sqlite3 myfirstdatabase.db
SQLite version 3.45.1 2024-01-30 16:01:20
Enter ".help" for usage hints.
sqlite>
```


DB Browser for SQLite - planets.db
File Edit View Tools Help
New Database
Open Database
Write Changes
Revert Changes
Open Project
Attach Database
Database Structure
Browse Data
Edit Pragmas
Execute SQL
Create Table
Create Index
Modify Table
Name
Type
Schema
Tables (3)
Landings
Planets
Satellites
Indices (0)
Views (0)
Triggers (0)
Edit Database Cell
Mode: Text
1 1
Type of data currently in cell: Text / Numeric
1 character
Apply
Remote
Identity
Select an identity to connect
DBHub.io
Local
Current Database
Name
Last modified
Size
SQL Log
Plot
DB Schema
Remote
UTF-8

SQL: **S**tructured **Q**uery **L**anguage

- 1970: Edgard Codd propone el **álgebra relacional** para bases de datos.



Edgard Codd

A Relational Model of Data
for Large Shared Data Banks

Communications of the ACM, Junio 1970.

SQL: **S**tructured **Q**uery **L**anguage

- 1970: Edgard Codd propone el **álgebra relacional** para bases de datos.
- 1974: Donald Chamberlin y Raymond Boyce proponen SEQUEL (IBM).

Donald Chamberlin

Raymond Boyce

SQL: **S**tructured **Q**uery **L**anguage

- 1970: Edgard Codd propone el **álgebra relacional** para bases de datos.
- 1974: Donald Chamberlin y Raymond Boyce proponen SEQUEL (IBM).
- 1986: Primer estándar (`SQL-86`) por ANSI.

$$\text{ANSI} = \text{American National Standards Institute}$$

Hay que pagar para acceder al estándar :(

SQL: **S**tructured **Q**uery **L**anguage

- 1970: Edgard Codd propone el **álgebra relacional** para bases de datos.
- 1974: Donald Chamberlin y Raymond Boyce proponen SEQUEL (IBM).
- 1986: Primer estándar (`SQL-86`) por ANSI.
- 1989: Revisión menor (`SQL-89`).
- 1992: Revisión substancial (`SQL-92` o `SQL2`).
- 1999: Nuevas características (`SQL-99` o `SQL3`).
  - Expresiones regulares
  - Recursión
  - Triggers

SQL: **S**tructured **Q**uery **L**anguage

- 1970: Edgard Codd propone el **álgebra relacional** para bases de datos.
- 1974: Donald Chamberlin y Raymond Boyce proponen SEQUEL (IBM).
- 1986: Primer estándar (`SQL-86`) por ANSI.
- 1989: Revisión menor (`SQL-89`).
- 1992: Revisión substancial (`SQL-92` o `SQL2`).
- 1999: Nuevas características (`SQL-99` o `SQL3`).
- 2003: Soporte de XML (`SQL:2003`)
- ...

Más usado/soportado es **SQL-92** con características de **SQL-99**

...casi ningún motor soporta todo el estándar.

SQL: **S**tructured **Q**uery **L**anguage

Sublenguajes dentro de SQL

1. Data Definition Language (DDL)
   - `CREATE` / `ALTER` / `DROP` / `TRUNCATE` / `RENAME TABLE`
2. Data Manipulation Language (DML)
   - `INSERT INTO` / `UPDATE` / `DELETE FROM` / `MERGE`
3. Data Query Language (DQL)
   - `SELECT`
4. Data Control Language (DCL)
   - `GRANT` / `REVOKE`

Esta clase veremos (parte de) DDL / DML / DQL

Outline

Modelo relacional sin atributos (recordatorio)

Sea **Data** un conjunto de **datos** **fijo**.

Definición

Un **esquema** (de bases de datos) es un par $\mathbb{S} = (\mathsf{N}, \mathsf{ar})$ tal que:

- N es un conjunto de **nombres** de relaciones y
- $\mathsf{ar} : \mathsf{N} \to \mathbb{N}$ es una función que asigna a cada nombre una **aridad**.

Definición

Sea $\mathbb{S} = (\mathsf{N}, \mathsf{ar})$ un esquema. Una **base de datos** D de $\mathbb{S}$ es una función:

$$D : \mathsf{N} \to \mathbf{Rel}$$

tal que $D(R) \subseteq \mathbf{Data}^k$ con $k = \mathsf{ar}(R)$.

Modelo relacional con atributos (nombrado)

Sea **Data** el conjunto de **datos** y **Att** un conjunto infinito de **atributos**.

Definición

Un **esquema** (nombrado) es un par $\mathbb{S} = (\mathsf{N}, \mathsf{ar})$ tal que:

- N es un conjunto de **nombres** de relaciones y
- $\mathsf{ar} : \mathsf{N} \to 2^{\mathbf{Att}}$ asigna a cada nombre un conjuntos finito de **atributos**.

Definición

Sea $\mathbb{S} = (\mathsf{N}, \mathsf{ar})$ un esquema. Una **base de datos** D de $\mathbb{S}$ es una función:

$$D : \mathsf{N} \to \mathbf{nRel}$$

tal que $D(R)$ es una nrelación con $\mathsf{att}(D(R)) = \mathsf{ar}(R)$.

DDL y **DML** nos permite definir el **esquema** $\mathbb{S}$ y la **base de datos** D

Ejemplo en ejecución: sistema planetario

Planets

name	dist	radio	grav	years	temp	ring
Mercurio	0.39	0.38	2.8	0.241	440	FALSE
Venus	0.72	0.95	8.9	0.615	730	FALSE
Tierra	1.00	1.00	9.8	1.000	288	FALSE
Marte	1.52	0.53	3.7	1.880	186	FALSE
Júpiter	5.20	10.97	22.9	11.862	152	TRUE
Saturno	9.54	9.14	9.1	29.447	134	TRUE
Urano	19.19	3.98	7.8	84.017	76	TRUE
Neptuno	30.07	3.86	11.0	164.791	53	TRUE

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Triton	Neptuno	Lassell	1845

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

(ejemplo de Aidan Hogan – UChile)

DDL: Crear relaciones

```
CREATE TABLE <nombre-rel> ( <nombre-att1>   <tipo1>,
                            <nombre-att2>   <tipo2>,
                            ...             ...
                            <nombre-attk>   <tipok> )
```

Ejemplo de planetas

```
CREATE TABLE Planets ( name  CHAR(20), dist   FLOAT, radio  FLOAT,
                       grav  FLOAT,    years  FLOAT, temp   INT,
                       ring  BOOLEAN )

CREATE TABLE Satellites ( name     CHAR(20), planet  CHAR(20),
                          discover CHAR(20), dyear   INT)

CREATE TABLE Landings ( spaceship CHAR(20), planet  CHAR(20),
                        ctry      CHAR(20), year    INT)
```

```
cristian@CRJ-X13-Yoga:~/IIC2413$
```

```
cristian@CRJ-X13-Yoga:~/IIC2413$ sqlite3 planets.db
```

```
cristian@CRJ-X13-Yoga:~/IIC2413$ sqlite3 planets.db
SQLite version 3.45.1 2024-01-30 16:01:20
Enter ".help" for usage hints.
sqlite>
```

```
cristian@CRJ-X13-Yoga:~/IIC2413$ sqlite3 planets.db
SQLite version 3.45.1 2024-01-30 16:01:20
Enter ".help" for usage hints.
sqlite> CREATE TABLE Planets (name CHAR(20), dist FLOAT, radio FLOAT, grav FLOAT,
 years FLOAT, temp INT, ring BOOLEAN);
```

```
cristian@CRJ-X13-Yoga:~/IIC2413$ sqlite3 planets.db
SQLite version 3.45.1 2024-01-30 16:01:20
Enter ".help" for usage hints.
sqlite> CREATE TABLE Planets (name CHAR(20), dist FLOAT, radio FLOAT, grav FLOAT,
 years FLOAT, temp INT, ring BOOLEAN);
sqlite>
```

```
cristian@CRJ-X13-Yoga:~/IIC2413$ sqlite3 planets.db
SQLite version 3.45.1 2024-01-30 16:01:20
Enter ".help" for usage hints.
sqlite> CREATE TABLE Planets (name CHAR(20), dist FLOAT, radio FLOAT, grav FLOAT,
 years FLOAT, temp INT, ring BOOLEAN);
sqlite> CREATE TABLE Satellites (name CHAR(20), planet CHAR(20), discover CHAR(20
), dyear INT);
```

```
cristian@CRJ-X13-Yoga:~/IIC2413$ sqlite3 planets.db
SQLite version 3.45.1 2024-01-30 16:01:20
Enter ".help" for usage hints.
sqlite> CREATE TABLE Planets (name CHAR(20), dist FLOAT, radio FLOAT, grav FLOAT,
 years FLOAT, temp INT, ring BOOLEAN);
sqlite> CREATE TABLE Satellites (name CHAR(20), planet CHAR(20), discover CHAR(20
), dyear INT);
sqlite>
```

```
cristian@CRJ-X13-Yoga:~/IIC2413$ sqlite3 planets.db
SQLite version 3.45.1 2024-01-30 16:01:20
Enter ".help" for usage hints.
sqlite> CREATE TABLE Planets (name CHAR(20), dist FLOAT, radio FLOAT, grav FLOAT,
 years FLOAT, temp INT, ring BOOLEAN);
sqlite> CREATE TABLE Satellites (name CHAR(20), planet CHAR(20), discover CHAR(20
), dyear INT);
sqlite> CREATE TABLE Landings (spaceship CHAR(20), planet CHAR(20), ctry CHAR(20)
, year INT);
```

```
cristian@CRJ-X13-Yoga:~/IIC2413$ sqlite3 planets.db
SQLite version 3.45.1 2024-01-30 16:01:20
Enter ".help" for usage hints.
sqlite> CREATE TABLE Planets (name CHAR(20), dist FLOAT, radio FLOAT, grav FLOAT,
 years FLOAT, temp INT, ring BOOLEAN);
sqlite> CREATE TABLE Satellites (name CHAR(20), planet CHAR(20), discover CHAR(20
), dyear INT);
sqlite> CREATE TABLE Landings (spaceship CHAR(20), planet CHAR(20), ctry CHAR(20)
, year INT);
sqlite>
```

```
cristian@CRJ-X13-Yoga:~/IIC2413$ sqlite3 planets.db
SQLite version 3.45.1 2024-01-30 16:01:20
Enter ".help" for usage hints.
sqlite> CREATE TABLE Planets (name CHAR(20), dist FLOAT, radio FLOAT, grav FLOAT,
 years FLOAT, temp INT, ring BOOLEAN);
sqlite> CREATE TABLE Satellites (name CHAR(20), planet CHAR(20), discover CHAR(20
), dyear INT);
sqlite> CREATE TABLE Landings (spaceship CHAR(20), planet CHAR(20), ctry CHAR(20)
, year INT);
sqlite> .tables
```

```
cristian@CRJ-X13-Yoga:~/IIC2413$ sqlite3 planets.db
SQLite version 3.45.1 2024-01-30 16:01:20
Enter ".help" for usage hints.
sqlite> CREATE TABLE Planets (name CHAR(20), dist FLOAT, radio FLOAT, grav FLOAT,
 years FLOAT, temp INT, ring BOOLEAN);
sqlite> CREATE TABLE Satellites (name CHAR(20), planet CHAR(20), discover CHAR(20
), dyear INT);
sqlite> CREATE TABLE Landings (spaceship CHAR(20), planet CHAR(20), ctry CHAR(20)
, year INT);
sqlite> .tables
Landings    Planets     Satellites
sqlite>
```

DDL: Crear relaciones

```
CREATE TABLE <nombre-rel> ( <nombre-att1>   <tipo1>,
                            <nombre-att2>   <tipo2>,
                            ...             ...
                            <nombre-attk>   <tipok> )
```

Definición (recordatorio)

Un **esquema** (nombrado) es un par $\mathbb{S} = (\mathsf{N}, \mathsf{ar})$ tal que:

- N es un conjunto de **nombres** de relaciones y
- $\mathsf{ar} : \mathsf{N} \to 2^{\mathbf{Att}}$ asigna a cada nombre un conjuntos finito de **atributos**.

`CREATE TABLE` $=$ $\mathbb{S}$ $+$ tipos de datos

DDL: Crear relaciones

```
CREATE TABLE <nombre-rel> ( <nombre-att1>   <tipo1>,
                            <nombre-att2>   <tipo2>,
                            ...             ...
                            <nombre-attk>   <tipok> )
```

Tipos de datos en SQL

- Numéricos: `INT`, `FLOAT`, `SMALL INT`, ...
- Strings caracteres: `CHAR(k)`, `VARCHAR(k)`, `CLOB`, ...
- Strings binarios: `BLOB`
- Booleanos: `BOOLEAN`
- Tiempo: `DATE`, `TIME`, `TIMESTAMP`, ...

Ver tipos de datos **soportados** por cada RDBMS.

DDL: Modificar esquema

Eliminar relación

```
DROP TABLE <nombre-rel>
```

Eliminar atributo

```
ALTER TABLE <nombre-rel> DROP <nombre-att>
```

Agregar atributo

```
ALTER TABLE <nombre-rel> ADD COLUMN <nombre-att>
```

En general, estos comandos son usados pocas veces

¿por qué?

Outline

DML: Insertar tuplas

no es necesario todos los atributos

```
INSERT INTO <nombre-rel>(<nombre-att1>, ..., <nombre-attk>)
VALUES (<valor1>, ..., <valork>)
```

Ejemplo de planetas

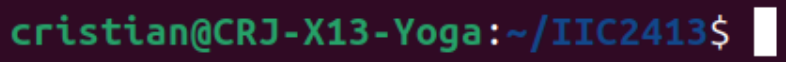
```
INSERT INTO Planets(name, dist, radio, grav, years, temp, ring)
VALUES ('Venus', 0.72, 0.95, 8.9, 0.615, 730, FALSE)

INSERT INTO Satellites(name, planet, discover, dyear)
VALUES ('Titán', 'Saturno', 'Huygens', 1655)

INSERT INTO Satellites(name, planet)
VALUES ('Luna', 'Tierra')
```

Los atributos no nombrados serán valores **NULL**

(ya veremos **NULL** más adelante)


cristian@CRJ-X13-Yoga:~/IIC2413$

```
cristian@CRJ-X13-Yoga:~/IIC2413$ sqlite3 planets.db
```

```
cristian@CRJ-X13-Yoga:~/IIC2413$ sqlite3 planets.db
SQLite version 3.45.1 2024-01-30 16:01:20
Enter ".help" for usage hints.
sqlite>
```

```
cristian@CRJ-X13-Yoga:~/IIC2413$ sqlite3 planets.db
SQLite version 3.45.1 2024-01-30 16:01:20
Enter ".help" for usage hints.
sqlite> INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES (
'Mercurio', 0.39, 0.38, 2.8, 0.241, 440, FALSE);
```

```
cristian@CRJ-X13-Yoga:~/IIC2413$ sqlite3 planets.db
SQLite version 3.45.1 2024-01-30 16:01:20
Enter ".help" for usage hints.
sqlite> INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES (
'Mercurio', 0.39, 0.38, 2.8, 0.241, 440, FALSE);
sqlite> INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES (
'Venus', 0.72, 0.95, 8.9, 0.615, 730, FALSE);
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Tierra'
, 1.00, 1.00, 9.8, 1.000, 288, FALSE);
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Marte',
 1.52, 0.53, 3.7, 1.880, 186, FALSE);
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Júpiter
', 5.20, 10.97, 22.9, 11.862, 152, TRUE);
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Saturno
', 9.54, 9.14, 9.1, 29.447, 134, TRUE);
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Urano',
 19.19, 3.98, 7.8, 84.017, 76, TRUE);
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Neptuno
', 30.07, 3.86, 11.0, 164.791, 53, TRUE);
```

```
cristian@CRJ-X13-Yoga:~/IIC2413$ sqlite3 planets.db
SQLite version 3.45.1 2024-01-30 16:01:20
Enter ".help" for usage hints.
sqlite> INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES (
'Mercurio', 0.39, 0.38, 2.8, 0.241, 440, FALSE);
sqlite> INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES (
'Venus', 0.72, 0.95, 8.9, 0.615, 730, FALSE);
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Tierra'
, 1.00, 1.00, 9.8, 1.000, 288, FALSE);
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Marte',
 1.52, 0.53, 3.7, 1.880, 186, FALSE);
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Júpiter
', 5.20, 10.97, 22.9, 11.862, 152, TRUE);
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Saturno
', 9.54, 9.14, 9.1, 29.447, 134, TRUE);
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Urano',
 19.19, 3.98, 7.8, 84.017, 76, TRUE);
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Neptuno
', 30.07, 3.86, 11.0, 164.791, 53, TRUE);
sqlite> INSERT INTO Satellites (name, planet, discover, dyear) VALUES ('Luna', 'T
ierra', NULL, NULL);
```

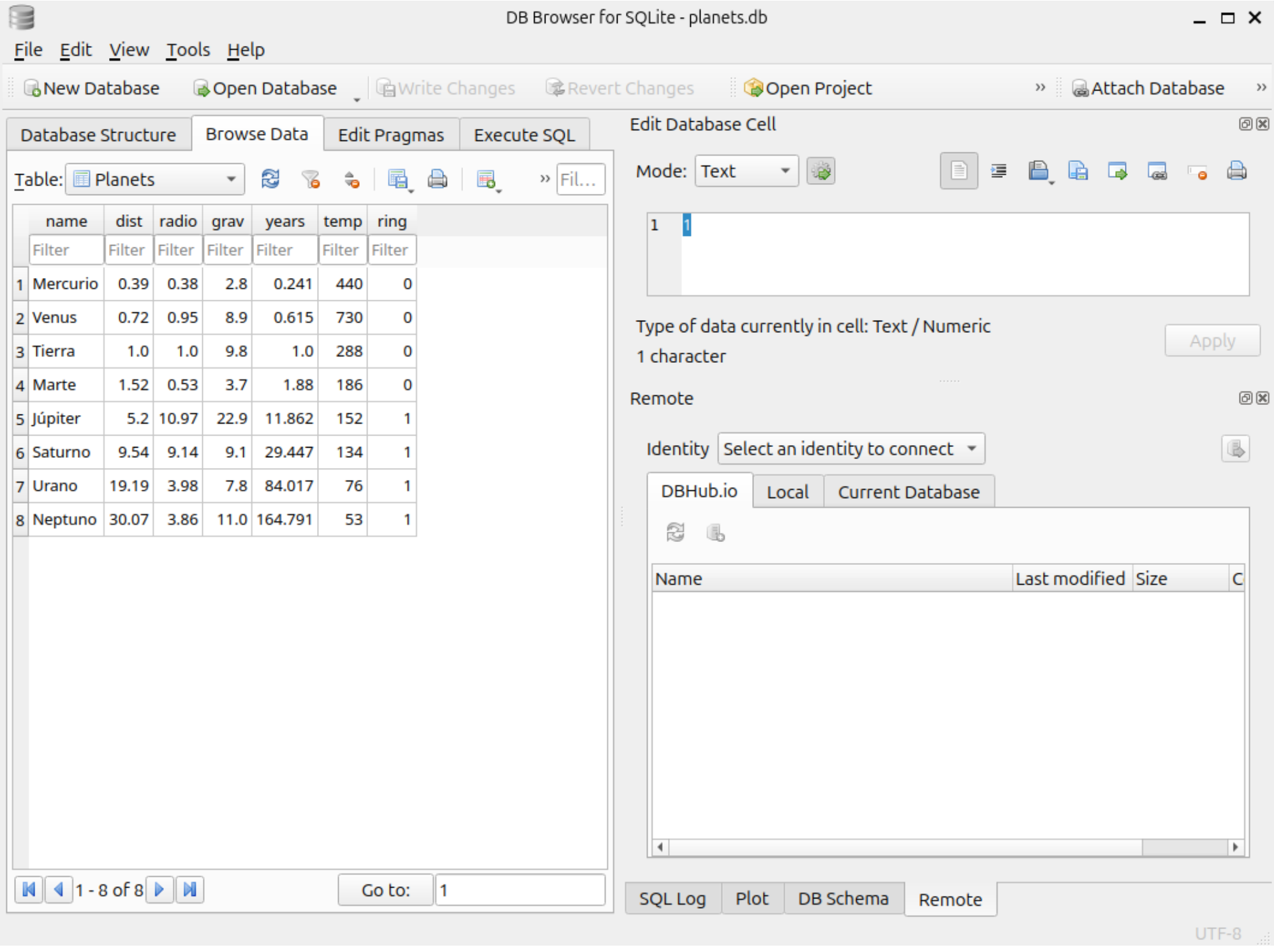
```
cristian@CRJ-X13-Yoga:~/IIC2413$ sqlite3 planets.db
SQLite version 3.45.1 2024-01-30 16:01:20
Enter ".help" for usage hints.
sqlite> INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES (
'Mercurio', 0.39, 0.38, 2.8, 0.241, 440, FALSE);
sqlite> INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES (
'Venus', 0.72, 0.95, 8.9, 0.615, 730, FALSE);
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Tierra'
, 1.00, 1.00, 9.8, 1.000, 288, FALSE);
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Marte',
 1.52, 0.53, 3.7, 1.880, 186, FALSE);
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Júpiter
', 5.20, 10.97, 22.9, 11.862, 152, TRUE);
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Saturno
', 9.54, 9.14, 9.1, 29.447, 134, TRUE);
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Urano',
 19.19, 3.98, 7.8, 84.017, 76, TRUE);
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Neptuno
', 30.07, 3.86, 11.0, 164.791, 53, TRUE);
sqlite> INSERT INTO Satellites (name, planet, discover, dyear) VALUES ('Luna', 'T
ierra', NULL, NULL);
sqlite>
```

```
'Venus', 0.72, 0.95, 8.9, 0.615, 730, FALSE);
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Tierra'
, 1.00, 1.00, 9.8, 1.000, 288, FALSE);
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Marte',
 1.52, 0.53, 3.7, 1.880, 186, FALSE);
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Júpiter
', 5.20, 10.97, 22.9, 11.862, 152, TRUE);
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Saturno
', 9.54, 9.14, 9.1, 29.447, 134, TRUE);
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Urano',
 19.19, 3.98, 7.8, 84.017, 76, TRUE);
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Neptuno
', 30.07, 3.86, 11.0, 164.791, 53, TRUE);
sqlite> INSERT INTO Satellites (name, planet, discover, dyear) VALUES ('Luna', 'T
ierra', NULL, NULL);
sqlite> INSERT INTO Satellites (name, planet, discover, dyear) VALUES ('Ganímedes
', 'Júpiter', 'Galileo', 1610);
INSERT INTO Satellites (name, planet, discover, dyear) VALUES ('Calisto', 'Júpite
r', 'Galileo', 1610);
INSERT INTO Satellites (name, planet, discover, dyear) VALUES ('Europa', 'Júpiter
', 'Galileo', 1610);
INSERT INTO Satellites (name, planet, discover, dyear) VALUES ('Ío', 'Júpiter', '
Galileo', 1610);
INSERT INTO Satellites (name, planet, discover, dyear) VALUES ('Titán', 'Saturno'
, 'Huygens', 1655);
INSERT INTO Satellites (name, planet, discover, dyear) VALUES ('Triton', 'Neptuno
', 'Lassell', 1845);
```

```
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Tierra'
, 1.00, 1.00, 9.8, 1.000, 288, FALSE);
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Marte',
 1.52, 0.53, 3.7, 1.880, 186, FALSE);
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Júpiter
', 5.20, 10.97, 22.9, 11.862, 152, TRUE);
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Saturno
', 9.54, 9.14, 9.1, 29.447, 134, TRUE);
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Urano',
 19.19, 3.98, 7.8, 84.017, 76, TRUE);
INSERT INTO Planets (name, dist, radio, grav, years, temp, ring) VALUES ('Neptuno
', 30.07, 3.86, 11.0, 164.791, 53, TRUE);
sqlite> INSERT INTO Satellites (name, planet, discover, dyear) VALUES ('Luna', 'T
ierra', NULL, NULL);
sqlite> INSERT INTO Satellites (name, planet, discover, dyear) VALUES ('Ganímedes
', 'Júpiter', 'Galileo', 1610);
INSERT INTO Satellites (name, planet, discover, dyear) VALUES ('Calisto', 'Júpite
r', 'Galileo', 1610);
INSERT INTO Satellites (name, planet, discover, dyear) VALUES ('Europa', 'Júpiter
', 'Galileo', 1610);
INSERT INTO Satellites (name, planet, discover, dyear) VALUES ('Ío', 'Júpiter', '
Galileo', 1610);
INSERT INTO Satellites (name, planet, discover, dyear) VALUES ('Titán', 'Saturno'
, 'Huygens', 1655);
INSERT INTO Satellites (name, planet, discover, dyear) VALUES ('Triton', 'Neptuno
', 'Lassell', 1845);
sqlite>
```

```
ierra', NULL, NULL);
sqlite> INSERT INTO Satellites (name, planet, discover, dyear) VALUES ('Ganímedes
', 'Júpiter', 'Galileo', 1610);
INSERT INTO Satellites (name, planet, discover, dyear) VALUES ('Calisto', 'Júpite
r', 'Galileo', 1610);
INSERT INTO Satellites (name, planet, discover, dyear) VALUES ('Europa', 'Júpiter
', 'Galileo', 1610);
INSERT INTO Satellites (name, planet, discover, dyear) VALUES ('Ío', 'Júpiter', '
Galileo', 1610);
INSERT INTO Satellites (name, planet, discover, dyear) VALUES ('Titán', 'Saturno'
, 'Huygens', 1655);
INSERT INTO Satellites (name, planet, discover, dyear) VALUES ('Triton', 'Neptuno
', 'Lassell', 1845);
sqlite> INSERT INTO Landings (spaceship, planet, ctry, year) VALUES ('Messenger',
 'Mercurio', 'EEUU', 2015);
INSERT INTO Landings (spaceship, planet, ctry, year) VALUES ('Venera 3', 'Venus',
 'URSS', 1966);
INSERT INTO Landings (spaceship, planet, ctry, year) VALUES ('Pioneer', 'Venus',
'EEUU', 1978);
INSERT INTO Landings (spaceship, planet, ctry, year) VALUES ('Mars 2', 'Marte', '
URSS', 1971);
INSERT INTO Landings (spaceship, planet, ctry, year) VALUES ('Viking 1', 'Marte',
 'EEUU', 1976);
INSERT INTO Landings (spaceship, planet, ctry, year) VALUES ('Beagle 2', 'Marte',
 'ESA', 2003);
INSERT INTO Landings (spaceship, planet, ctry, year) VALUES ('Galileo', 'Júpiter'
, 'EEUU', 2003);
```

```
sqlite> INSERT INTO Satellites (name, planet, discover, dyear) VALUES ('Ganímedes
', 'Júpiter', 'Galileo', 1610);
INSERT INTO Satellites (name, planet, discover, dyear) VALUES ('Calisto', 'Júpite
r', 'Galileo', 1610);
INSERT INTO Satellites (name, planet, discover, dyear) VALUES ('Europa', 'Júpiter
', 'Galileo', 1610);
INSERT INTO Satellites (name, planet, discover, dyear) VALUES ('Ío', 'Júpiter', '
Galileo', 1610);
INSERT INTO Satellites (name, planet, discover, dyear) VALUES ('Titán', 'Saturno'
, 'Huygens', 1655);
INSERT INTO Satellites (name, planet, discover, dyear) VALUES ('Triton', 'Neptuno
', 'Lassell', 1845);
sqlite> INSERT INTO Landings (spaceship, planet, ctry, year) VALUES ('Messenger',
 'Mercurio', 'EEUU', 2015);
INSERT INTO Landings (spaceship, planet, ctry, year) VALUES ('Venera 3', 'Venus',
 'URSS', 1966);
INSERT INTO Landings (spaceship, planet, ctry, year) VALUES ('Pioneer', 'Venus',
'EEUU', 1978);
INSERT INTO Landings (spaceship, planet, ctry, year) VALUES ('Mars 2', 'Marte', '
URSS', 1971);
INSERT INTO Landings (spaceship, planet, ctry, year) VALUES ('Viking 1', 'Marte',
 'EEUU', 1976);
INSERT INTO Landings (spaceship, planet, ctry, year) VALUES ('Beagle 2', 'Marte',
 'ESA', 2003);
INSERT INTO Landings (spaceship, planet, ctry, year) VALUES ('Galileo', 'Júpiter'
, 'EEUU', 2003);
sqlite>
```


DB Browser for SQLite - planets.db
File Edit View Tools Help
New Database
Open Database
Write Changes
Revert Changes
Open Project
Attach Database
Database Structure
Browse Data
Edit Pragmas
Execute SQL
Table: Planets
Fil...
name dist radio grav years temp ring
Filter
1 Mercurio 0.39 0.38 2.8 0.241 440 0
2 Venus 0.72 0.95 8.9 0.615 730 0
3 Tierra 1.0 1.0 9.8 1.0 288 0
4 Marte 1.52 0.53 3.7 1.88 186 0
5 Júpiter 5.2 10.97 22.9 11.862 152 1
6 Saturno 9.54 9.14 9.1 29.447 134 1
7 Urano 19.19 3.98 7.8 84.017 76 1
8 Neptuno 30.07 3.86 11.0 164.791 53 1
1 - 8 of 8
Go to: 1
Edit Database Cell
Mode: Text
Type of data currently in cell: Text / Numeric
1 character
Apply
Remote
Identity
Select an identity to connect
DBHub.io
Local
Current Database
Name
Last modified
Size
SQL Log
Plot
DB Schema
Remote
UTF-8

DML: Insertar tuplas

```
INSERT INTO <nombre-rel>(<nombre-att1>, ..., <nombre-attk>)
VALUES (<valor1>, ..., <valork>)
```

(<nombre-att1>, ..., <nombre-attk>): no es necesario todos los atributos

Definición (recordatorio)

Sea $\mathbb{S} = (\mathsf{N}, \mathsf{ar})$ un esquema. Una **base de datos** D de $\mathbb{S}$ es una función:

$$D : \mathsf{N} \to \textbf{nRel}$$

tal que $D(R)$ es una nrelación con $\mathsf{att}(D(R)) = \mathsf{ar}(R)$.

`INSERT INTO R, ..., INSERT INTO R` $= D(R)$

DML: Actualizar y eliminar tuplas

Actualizar una tupla

```
UPDATE <nombre-rel>
SET <nombre-att1> = <valor1>, ..., <nombre-attk> = <valork>
WHERE <condicion>
```

Ejemplo de planetas

```
UPDATE Satellites
SET name = 'Tritón', dyear = 1846
WHERE name = 'Triton'
```

Usando `<condicion>` podemos actualizar varios valores **simultáneamente**.

DML: Actualizar y eliminar tuplas

Actualizar una tupla

```
UPDATE <nombre-rel>
SET <nombre-att1> = <valor1>, ..., <nombre-attk> = <valork>
WHERE <condicion>
```

Eliminar una tupla

```
DELETE FROM <nombre-rel>
WHERE <condicion>
```

Usando `<condicion>` podemos eliminar varias tuplas **simultáneamente**.

Outline

DQL: El lenguaje de consultas de SQL

```
Q   :=   SELECT [DISTINCT] <lista-att>
         FROM <lista-relaciones>
         WHERE <condicion>
     |   Q1 UNION Q2
     |   Q1 UNION ALL Q2
     |   Q1 INTERSECT Q2
     |   Q1 EXCEPT Q2
     |   Q1 EXCEPT ALL Q2
```

Caso base

Casos inductivos

- SQL es **declarativo**.

  "Decimos lo que queremos,
  pero sin dar detalles de cómo lo computamos"

- El motor RDBMS transforma la consulta en un algoritmo eficiente.

SQL basado en **álgebra relacional** / **lógica de primer orden**

DQL: Aprendiendo SQL con álgebra relacional

Álgebra relacional

1. R
2. Proyección $\pi_{B_1,\ldots,B_k}$
3. Selección $\sigma_{A\sim a}$, $\sigma_{A\sim B}$
4. Renombrar $\rho_{A\to B}$
5. Producto cruz $\times$
6. Join $\bowtie$
7. Operadores de conjuntos: $\cup$, $\cap$, $\setminus$

DQL: Acceder una relación R

Planets

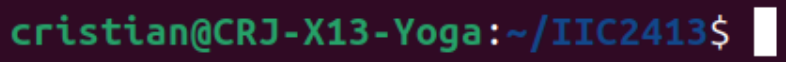
name	dist	radio	grav	years	temp	ring
Mercurio	0.39	0.38	2.8	0.241	440	FALSE
Venus	0.72	0.95	8.9	0.615	730	FALSE
Tierra	1.00	1.00	9.8	1.000	288	FALSE
Marte	1.52	0.53	3.7	1.880	186	FALSE
Júpiter	5.20	10.97	22.9	11.862	152	TRUE
Saturno	9.54	9.14	9.1	29.447	134	TRUE
Urano	19.19	3.98	7.8	84.017	76	TRUE
Neptuno	30.07	3.86	11.0	164.791	53	TRUE

Ejemplo: Planets

```
SELECT *
FROM Planets
```

$\Rightarrow$

name	dist	radio	grav	...
Mercurio	0.39	0.38	2.8	...
Venus	0.72	0.95	8.9	...
Tierra	1.00	1.00	9.8	...
Marte	1.52	0.53	3.7	...
Júpiter	5.20	10.97	22.9	...
Saturno	9.54	9.14	9.1	...
Urano	19.19	3.98	7.8	...
Neptuno	30.07	3.86	11.0	...


cristian@CRJ-X13-Yoga:~/IIC2413$

```
cristian@CRJ-X13-Yoga:~/IIC2413$ sqlite3 planets.db
```

```
cristian@CRJ-X13-Yoga:~/IIC2413$ sqlite3 planets.db
SQLite version 3.45.1 2024-01-30 16:01:20
Enter ".help" for usage hints.
sqlite>
```

```
cristian@CRJ-X13-Yoga:~/IIC2413$ sqlite3 planets.db
SQLite version 3.45.1 2024-01-30 16:01:20
Enter ".help" for usage hints.
sqlite> SELECT * FROM Planets;
```

```
cristian@CRJ-X13-Yoga:~/IIC2413$ sqlite3 planets.db
SQLite version 3.45.1 2024-01-30 16:01:20
Enter ".help" for usage hints.
sqlite> SELECT * FROM Planets;
Mercurio|0.39|0.38|2.8|0.241|440|0
Venus|0.72|0.95|8.9|0.615|730|0
Tierra|1.0|1.0|9.8|1.0|288|0
Marte|1.52|0.53|3.7|1.88|186|0
Júpiter|5.2|10.97|22.9|11.862|152|1
Saturno|9.54|9.14|9.1|29.447|134|1
Urano|19.19|3.98|7.8|84.017|76|1
Neptuno|30.07|3.86|11.0|164.791|53|1
sqlite>
```

DQL: Acceder una relación R

Planets

name	dist	radio	grav	years	temp	ring
Mercurio	0.39	0.38	2.8	0.241	440	FALSE
Venus	0.72	0.95	8.9	0.615	730	FALSE
Tierra	1.00	1.00	9.8	1.000	288	FALSE
Marte	1.52	0.53	3.7	1.880	186	FALSE
Júpiter	5.20	10.97	22.9	11.862	152	TRUE
Saturno	9.54	9.14	9.1	29.447	134	TRUE
Urano	19.19	3.98	7.8	84.017	76	TRUE
Neptuno	30.07	3.86	11.0	164.791	53	TRUE

Ejemplo: Planets

```
SELECT *
FROM Planets
```

$\equiv \quad Q(x_{\text{name}}, x_{\text{dist}}, x_{\text{radio}}, x_{\text{grac}}, x_{\text{years}}, x_{\text{temp}}, x_{\text{ring}}) \leftarrow$
$\text{Planets}(x_{\text{name}}, x_{\text{dist}}, x_{\text{radio}}, x_{\text{grac}}, x_{\text{years}}, x_{\text{temp}}, x_{\text{ring}})$

¿cómo escribimos esta consulta en lógica de primer orden?

DQL: Aprendiendo SQL con álgebra relacional

Álgebra relacional

1. R ✓
2. Proyección $\pi_{B_1,\ldots,B_k}$
3. Selección $\sigma_{A\sim a}$, $\sigma_{A\sim B}$
4. Renombrar $\rho_{A\to B}$
5. Producto cruz $\times$
6. Join $\bowtie$
7. Operadores de conjuntos: $\cup$, $\cap$, $\setminus$

DQL: Proyección $\pi_{B_1,\ldots,B_k}$

Planets

name	dist	radio	grav	years	temp	ring
Mercurio	0.39	0.38	2.8	0.241	440	FALSE
Venus	0.72	0.95	8.9	0.615	730	FALSE
Tierra	1.00	1.00	9.8	1.000	288	FALSE
Marte	1.52	0.53	3.7	1.880	186	FALSE
Júpiter	5.20	10.97	22.9	11.862	152	TRUE
Saturno	9.54	9.14	9.1	29.447	134	TRUE
Urano	19.19	3.98	7.8	84.017	76	TRUE
Neptuno	30.07	3.86	11.0	164.791	53	TRUE

Ejemplo: $\pi_{\mathbf{name},\mathbf{years}}(\text{Planets})$

```
SELECT name, years
FROM Planets
```

$\Rightarrow$

name	years
Mercurio	0.241
Venus	0.615
Tierra	1.000
Marte	1.880
Júpiter	11.862
Saturno	29.447
Urano	84.017
Neptuno	164.791

DQL: Proyección $\pi_{B_1,\ldots,B_k}$

Planets

name	dist	radio	grav	years	temp	ring
Mercurio	0.39	0.38	2.8	0.241	440	FALSE
Venus	0.72	0.95	8.9	0.615	730	FALSE
Tierra	1.00	1.00	9.8	1.000	288	FALSE
Marte	1.52	0.53	3.7	1.880	186	FALSE
Júpiter	5.20	10.97	22.9	11.862	152	TRUE
Saturno	9.54	9.14	9.1	29.447	134	TRUE
Urano	19.19	3.98	7.8	84.017	76	TRUE
Neptuno	30.07	3.86	11.0	164.791	53	TRUE

Ejemplo: $\pi_{\textbf{name},\textbf{years}}(\text{Planets})$

```
SELECT name, years
FROM Planets
```

$\equiv \quad Q(x_{\text{name}}, x_{\text{years}}) \leftarrow$

$\exists x \exists y \exists z \exists u \exists v.$

$\text{Planets}(x_{\text{name}}, x, y, z, x_{\text{years}}, u, v)$

¿cómo escribimos esta consulta en lógica de primer orden?

DQL: Proyección $\pi_{B_1,\ldots,B_k}$

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo: $\pi_{\textbf{planet}}(\text{Landings})$

```
SELECT planet
FROM Landings
```

$\Rightarrow$

planet
Mercurio
Venus
Venus
Marte
Marte
Marte
Júpiter

(¿tenemos un problema?)

DQL: Proyección $\pi_{B_1,\ldots,B_k}$

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

SQL cambia algunas reglas del álgebra relacional

- Permite duplicados.
  - **Proyección** en SQL permite duplicados.
  - Relaciones pueden tener duplicados inicialmente.
- Permite orden de las tuplas.
  - En los resultados y visualización (lo veremos más adelante).

¿por qué SQL **no sigue** los modelos teóricos?

Time to Move on: Querying without Nulls and Bags

Molham Aref
RelationalAI

Leonid Libkin
RelationalAI
University of Edinburgh

Wim Martens
RelationalAI
University of Bayreuth

Abstract

SQL is *the* database community's success story in terms of language design. The key reason for its success is its declarativeness: it gives rise to optimizability, reducing the programmer's burden significantly. However, given the evolving complexity of problems to solve with query languages, our community needs to re-think some of the fundamental early query language design decisions.

Our experience in having worked on the design of Rel (a language for end-to-end relational programming) tells us that it is possible to design, implement, and successfully deploy a language based on fully normalized relations. Such relations avoid what Codd called *corrupted relations* and what we commonly refer to as *bags*, and the "harmful" *billion dollar mistake* that we know as *nulls*. In the SQL world, it is accepted that bags and nulls are tolerated as an unavoidable evil. We argue that the evil is completely avoidable: reasons offered for justifying bags and nulls evaporate at a closer examination. In addition to debunking them, we also describe opportunities offered by a null-free language with set semantics.

Keywords

Declarative languages, SQL, Nulls, Bags

ACM Reference Format:
Molham Aref, Leonid Libkin, and Wim Martens. 2026. Time to Move on: Querying without Nulls and Bags. In *Proceedings of (...)*. ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

for modifying and improving SQL have also emerged. In fact, early ideas of replacing SQL (or at least proposing an alternative to it) are already found in Date and Darwen's Tutorial D [12]. This trend continued later with more modern proposals. Some of them, like [27, 28], try to sequentialize SQL and bring it closer to imperative programming; others, like Rel [2], SaneQL [25] fully adhere to the principle of declarativeness.

A key criticism of SQL is the extremely high complexity of SQL Standard. This complexity is inherent in its design as a *data sublanguage* [9]. That is to say, it is a declarative sublanguage for handling operations with data, with the rest delegated to a general purpose programming language. This approach made perfect sense back in the 1970s and 1980s, when declarative programming had to win over alternative proposals: going fully declarative at once was too ambitious. In fact, it is still an unrealized dream expressed by Jim Gray as the holy grail of *automatic programming* [18], where the goal is to design a high-level language sufficiently powerful for specifying *all application tasks*.

SQL's sublanguage design, however, has led to problems. SQL lacks facilities for writing libraries, which entails that every new desired feature has to be added to the language itself, and therefore become part of the standard. Today, the core of the SQL Standard (SQL Foundation) is over 1,500 pages, and the entire standard is well over 4,000 pages. For comparison, the C standard is an order of magnitude smaller. A language that needs to be syntactically extended for each new functionality ends up with hundreds of reserved keywords.

This makes queries error-prone: the more complex a language is,

DQL: Proyección $\pi_{B_1,\ldots,B_k}$

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo: $\pi_{\textbf{planet}}$(Landings) (continuación)

```
SELECT DISTINCT planet
FROM Landings
```

$\Rightarrow$

planet
Mercurio
Venus
Marte
Júpiter

DISTINCT nos permite declarar la **eliminación de duplicados**

DQL: Aprendiendo SQL con álgebra relacional

Álgebra relacional

1. R ✓
2. Proyección $\pi_{B_1,\ldots,B_k}$ ✓
3. Selección $\sigma_{A\sim a}$, $\sigma_{A\sim B}$
4. Renombrar $\rho_{A\to B}$
5. Producto cruz $\times$
6. Join $\bowtie$
7. Operadores de conjuntos: $\cup$, $\cap$, $\setminus$

DQL: Selección $\sigma_{A\sim a}$, $\sigma_{A\sim B}$

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo: $\pi_{\textbf{spaceship}}(\sigma_{\textbf{ctry}=\text{`EEUU'}}(\text{Landings}))$

```
SELECT spaceship
FROM Landings
WHERE ctry = 'EEUU'
```

$\Rightarrow$

spaceship
Messenger
Pioneer
Viking 1
Galileo

DQL: Selección $\sigma_{A\sim a}$, $\sigma_{A\sim B}$

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo: $\pi_{\textbf{spaceship}}(\sigma_{\textbf{ctry}=\text{`EEUU'}}(\text{Landings}))$

```
SELECT spaceship
FROM Landings
WHERE ctry = 'EEUU'
```

$\equiv$

$$Q(x_{\text{spaceship}}) \leftarrow \exists x \exists y \exists z.\ \text{Landings}(x_{\text{spaceship}}, x, y, z) \overbrace{\wedge\ y = \text{EEUU}}^{\texttt{WHERE}}$$

¿en lógica de primer orden?

DQL: Selección σ_θ

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo: $\pi_{\textbf{spaceship}}(\sigma_{\textbf{ctry}=\text{`EEUU'} \wedge \textbf{year}>2000}(\text{Landings}))$

```
SELECT spaceship
FROM Landings
WHERE ctry = 'EEUU' AND year > 2000
```

$\Rightarrow$

spaceship
Messenger
Galileo

En `WHERE` podemos usar los **conectivos lógicos** `AND`, `OR`, y `NOT`

DQL: Selección + Proyección

SQL

```
SELECT [DISTINCT] <lista-att>
FROM <relacion>
WHERE <condicion>
```

Algebra relacional

$$\pi_{\texttt{<lista-att>}}\Big(\ \sigma_{\texttt{<condicion>}}\big(\underbrace{R}_{\texttt{<relacion>}}\big)\ \Big)$$

Consultas de primer orden

$$\underbrace{Q(\ \bar{x}\ )}_{\texttt{<lista-att>}}\ \leftarrow\ \exists \bar{y}.\ \underbrace{R}_{\texttt{<relacion>}}(\bar{x},\bar{y})\ \wedge\ \underbrace{\theta}_{\texttt{<condicion>}}$$

DQL: Aprendiendo SQL con álgebra relacional

Álgebra relacional

1. R ✓
2. Proyección $\pi_{B_1,\ldots,B_k}$ ✓
3. Selección $\sigma_{A\sim a}$, $\sigma_{A\sim B}$ ✓
4. Renombrar $\rho_{A\to B}$
5. Producto cruz $\times$
6. Join $\bowtie$
7. Operadores de conjuntos: $\cup$, $\cap$, $\setminus$

DQL: Renombrar $\rho_{A \to B}$

Planets

name	dist	radio	grav	years	temp	ring
Mercurio	0.39	0.38	2.8	0.241	440	FALSE
Venus	0.72	0.95	8.9	0.615	730	FALSE
Tierra	1.00	1.00	9.8	1.000	288	FALSE
Marte	1.52	0.53	3.7	1.880	186	FALSE
Júpiter	5.20	10.97	22.9	11.862	152	TRUE
Saturno	9.54	9.14	9.1	29.447	134	TRUE
Urano	19.19	3.98	7.8	84.017	76	TRUE
Neptuno	30.07	3.86	11.0	164.791	53	TRUE

Ejemplo: $\rho_{\mathbf{grav} \to \mathbf{g}}(\pi_{\mathbf{grav},\mathbf{name}}(\sigma_{\mathbf{grav}>9.8}(\text{Planets})))$

```
SELECT grav AS g, name
FROM Planets
WHERE grav > 9.8
```

$\Rightarrow$

g	name
22.9	Júpiter
11.0	Neptuno

Notar que **orden de atributos** define el **orden de la visualización**

DQL: Aprendiendo SQL con álgebra relacional

Álgebra relacional

1. R ✓
2. Proyección $\pi_{B_1,\ldots,B_k}$ ✓
3. Selección $\sigma_{A\sim a}$, $\sigma_{A\sim B}$ ✓
4. Renombrar $\rho_{A\to B}$ ✓
5. Producto cruz $\times$
6. Join $\bowtie$
7. Operadores de conjuntos: $\cup$, $\cap$, $\setminus$

DQL: Producto cruz $\times$

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Tritón	Neptuno	Lassell	1846

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo: $\pi_{\textbf{name}, \textbf{planet}, \textbf{spaceship}}(\text{Satellites} \times \text{Landings})$

```
SELECT name, planet, spaceship
FROM Satellites, Landings
```

$\Rightarrow$

name	planet	spaceship
Luna	?	Messenger
Luna	?	Venera3
...	...	...
Ganímedes	?	Messenger
Ganímedes	?	Venera3
...	...	...

¿cuál **planet** escoge SQL?

DQL: Producto cruz $\times$

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Tritón	Neptuno	Lassell	1846

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo: $\pi_{\textbf{name},\, \textbf{S.planet},\, \textbf{spaceship}}(\rho_{\textbf{planet}\rightarrow\textbf{S.planet}}(\text{Satellites}) \times \text{Landings})$

```
SELECT name, S.planet, spaceship
FROM Satellites AS S, Landings
```

$\Longrightarrow$

name	planet	spaceship
Luna	Tierra	Messenger
Luna	Tierra	Venera3
...	...	...
Ganímedes	Júpiter	Messenger
Ganímedes	Júpiter	Venera3
...	...	...

"`AS S`" en `FROM` **renombra cada atributo** `A` como `S.A`

DQL: Producto cruz $\times$

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Tritón	Neptuno	Lassell	1846

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo: $\pi_{\textbf{name},\,\textbf{S.planet},\,\textbf{spaceship}}(\rho_{\textbf{planet}\to\textbf{S.planet}}(\text{Satellites}) \times \text{Landings})$

```
SELECT name, S.planet, spaceship
FROM Satellites AS S, Landings
```

$\equiv$

$$Q(x_{\text{name}}, x_{\text{planet}}, x_{\text{spaceship}}) \leftarrow$$
$$\exists x \exists y \exists z \exists u \exists v.\ \text{Satellites}(x_{\text{name}}, x_{\text{planet}}, x, y) \wedge \text{Landings}(x_{\text{spaceship}}, z, u, v)$$

¿cómo se escribe en lógica de primer orden?

DQL: Aprendiendo SQL con álgebra relacional

Álgebra relacional

1. R ✓
2. Proyección $\pi_{B_1,\ldots,B_k}$ ✓
3. Selección $\sigma_{A\sim a}$, $\sigma_{A\sim B}$ ✓
4. Renombrar $\rho_{A\to B}$ ✓
5. Producto cruz $\times$ ✓
6. Join $\bowtie$
7. Operadores de conjuntos: $\cup$, $\cap$, $\setminus$

DQL: Join $\bowtie$ (recordatorio)

Si $\bar{\tau}_1$ y $\bar{\tau}_2$ son **compatibles** ($\bar{\tau}_1 \sim \bar{\tau}_2 : \forall A \in \text{att}(\bar{\tau}_1) \cap \text{att}(\bar{\tau}_2).\ \bar{\tau}_1(A) = \bar{\tau}_2(A)$):

$$[\bar{\tau}_1 \bowtie \bar{\tau}_2](A) := \begin{cases} \bar{\tau}_1(A) & \text{si } A \in \text{att}(\bar{\tau}_1) \\ \bar{\tau}_2(A) & \text{si } A \in \text{att}(\bar{\tau}_2) \end{cases}$$

Definición

Para **relaciones** $\mathcal{R}_1$ y $\mathcal{R}_2$ se define el **join**:

$$\mathcal{R}_1 \bowtie \mathcal{R}_2 := \{ \bar{\tau}_1 \bowtie \bar{\tau}_2 \mid \bar{\tau}_1 \in \mathcal{R}_1 \wedge \bar{\tau}_2 \in \mathcal{R}_2 \wedge \bar{\tau}_1 \sim \bar{\tau}_2 \}$$

Ejemplo de $\bowtie$

pid	name	cid
1	Ana	101
2	Ben	102
3	Zoe	102

$\bowtie$

pid	job
1	Guía
2	Chef
2	Guía
2	Poeta

$=$

pid	name	cid	job
1	Ana	101	Guía
2	Ben	102	Chef
2	Ben	102	Guía
2	Ben	102	Poeta

DQL: Join $\bowtie$

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Tritón	Neptuno	Lassell	1846

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo: $\pi_{\textbf{name}, \textbf{planet}}(\text{Satellites} \bowtie \text{Landings})$

```
SELECT name, planet
FROM Satellites
   NATURAL JOIN Landings
```

$\Rightarrow$

name	planet
Ganímedes	Júpiter
Calisto	Júpiter
Europa	Júpiter
Ío	Júpiter

En general, este es el **operador de join** en SQL **menos usado** :(

DQL: Join ⋈

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Tritón	Neptuno	Lassell	1846

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo: $\pi_{\textbf{name}, \textbf{planet}}(\text{Satellites} \bowtie \text{Landings})$

```
SELECT name, planet
FROM Satellites
   JOIN Landings USING (planet)
```

$\Rightarrow$

name	planet
Ganímedes	Júpiter
Calisto	Júpiter
Europa	Júpiter
Ío	Júpiter

Un **poco más usado** en SQL (pero no el más usado) :(

DQL: Join $\bowtie$

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Tritón	Neptuno	Lassell	1846

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Teorema (recordatorio)

Para $\mathcal{R}_1$ y $\mathcal{R}_2$ con $\text{att}(\mathcal{R}_1) = \{A, B\}$ y $\text{att}(\mathcal{R}_2) = \{B, C\}$ se tiene que:

$$\mathcal{R}_1 \bowtie \mathcal{R}_2 \;=\; \pi_{A,B,C}\Big(\sigma_{B=B'}\big(\mathcal{R}_1 \times \rho_{B\to B'}(\mathcal{R}_2)\big)\Big)$$

Siempre podemos hacer un join con **producto cruz** y **selección**.

DQL: Join $\bowtie$

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Tritón	Neptuno	Lassell	1846

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo: $\pi_{\textbf{name}, \textbf{planet}}(\text{Satellites} \bowtie \text{Landings})$

```
SELECT name, S.planet
FROM Satellites AS S, Landings AS L
WHERE S.planet = L.planet
```

$\Rightarrow$

name	planet
Ganímedes	Júpiter
Calisto	Júpiter
Europa	Júpiter
Ío	Júpiter

Esta es la forma **más común** como los usuarios hacen un join.

DQL: Join $\bowtie$

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Tritón	Neptuno	Lassell	1846

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo: $\pi_{\textbf{name},\,\textbf{planet}}(\text{Satellites} \bowtie \text{Landings})$

```
SELECT name, S.planet                    ≡
FROM Satellites AS S, Landings AS L
WHERE S.planet = L.planet
```

$$Q(x_{\text{name}}, x_{\text{planet}}) \leftarrow \exists x \exists y \exists z \exists u \exists v . \exists w .$$
$$\text{Satellites}(x_{\text{name}}, x_{\text{planet}}, x, y) \wedge \text{Landings}(z, u, v, w) \wedge \overbrace{x_{\text{planet}} = u}^{\texttt{<condicion>}}$$

¿y en lógica de primer orden?

DQL: Join $\bowtie$

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Tritón	Neptuno	Lassell	1846

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo: $\pi_{\textbf{name},\,\textbf{planet}}(\text{Satellites} \bowtie \text{Landings})$

```
SELECT name, S.planet                    ≡
FROM Satellites AS S, Landings AS L
WHERE S.planet = L.planet
```

$$Q(x_{\text{name}}, x_{\text{planet}}) \leftarrow \exists x \exists y \exists z \exists v . \exists w .$$
$$\text{Satellites}(x_{\text{name}}, x_{\text{planet}}, x, y) \wedge \text{Landings}(z, x_{\text{planet}}, v, w)$$

¿y en lógica de primer orden?

DQL: Join $\bowtie$ + Selecciones

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Tritón	Neptuno	Lassell	1846

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo: $\pi_{\textbf{name},\,\textbf{planet}}(\sigma_{\textbf{dyear}>2000}(\text{Satellites} \bowtie \text{Landings}))$

```
SELECT name, S.planet
FROM Satellites AS S, Landings AS L
WHERE S.planet = L.planet
    AND dyear > 2000
```

$\Rightarrow$

name	planet

DQL: Selección + Proyección + Joins

SQL

```
SELECT [DISTINCT] <lista-att>
FROM <lista-relaciones>
WHERE <condicion>
```

Algebra relacional

$$\pi_{\texttt{<lista-att>}}\Big(\ \sigma_{\texttt{<condicion>}}\big(\ \underbrace{R_1 \times \cdots \times R_k}_{\texttt{<lista-relaciones>}}\big)\ \Big)$$

Consultas de primer orden

$$Q(\ \underbrace{\bar{x}}_{\texttt{<lista-att>}}\ ) \leftarrow \exists \bar{y}.\ \underbrace{R_1 \wedge \cdots \wedge R_k}_{\texttt{<lista-relaciones>}} \wedge \overbrace{\theta}^{\texttt{<condicion>}}$$

DQL: Aprendiendo SQL con álgebra relacional

Álgebra relacional

1. R ✓
2. Proyección $\pi_{B_1,\ldots,B_k}$ ✓
3. Selección $\sigma_{A\sim a}$, $\sigma_{A\sim B}$ ✓
4. Renombrar $\rho_{A\to B}$ ✓
5. Producto cruz $\times$ ✓
6. Join $\bowtie$ ✓
7. Operadores de conjuntos: $\cup$, $\cap$, $\setminus$

DQL: Unión $\cup$

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Tritón	Neptuno	Lassell	1846

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo: $\pi_{\textbf{planet}}(\text{Satellites}) \cup \pi_{\textbf{planet}}(\text{Landings})$

```
SELECT planet
FROM Satellites
UNION
SELECT planet
FROM Landings
```

$\Rightarrow$

planet
Tierra
Júpiter
Saturno
Neptuno
Mercurio
Venus
Marte

¿y donde están los duplicados?

DQL: Unión $\cup$

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Tritón	Neptuno	Lassell	1846

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

SQL y duplicados

`SELECT` $\Rightarrow$ Proyección con **duplicados**

`UNION` $\Rightarrow$ Unión de conjuntos

`UNION ALL` $\Rightarrow$ Unión con **duplicados**

DQL: Unión $\cup$

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Tritón	Neptuno	Lassell	1846

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo: $\pi_{\textbf{planet}}(\text{Satellites}) \cup \pi_{\textbf{planet}}(\text{Landings})$

```
SELECT planet
FROM Satellites
UNION ALL
SELECT planet
FROM Landings
```

$\Rightarrow$

planet
Tierra
Júpiter
Júpiter
Júpiter
Júpiter
Saturno
...

DQL: Operadores de conjuntos: $\cup$, $\cap$, $\setminus$

Unión $\cup$

`UNION`	$\Rightarrow$	Unión de conjuntos
`UNION ALL`	$\Rightarrow$	Unión con **duplicados**

Intersección $\cap$

`INTERSECT`	$\Rightarrow$	Intersección de conjuntos
`INTERSECT ALL`	$\Rightarrow$	Intersección con **duplicados**

Diferencia $\setminus$

`EXCEPT`	$\Rightarrow$	Diferencia de conjuntos
`EXCEPT ALL`	$\Rightarrow$	Diferencia con **duplicados**